Deepspace Defenders — Space Shooter Game

A Semester Project Report

Course:	CSE142 Object Oriented Programming Techniques
Semester:	Spring 2026
Instructor:	Behraj Khan
Due Date:	10th May 2026
Author(s):	Hamza Faisal — ERP: 32406
	Hunain Adnan — ERP: 32397
	Burhanuddin Murtaza Patanwala — ERP: 32400

Department of Computer Science
Institute of Business Administration, Karachi

May 10, 2026

Abstract

Deepspace Defenders is a fully playable 2D space shooter game developed in C++ using the SFML (Simple and Fast Multimedia Library) graphics framework. The player controls a spaceship and must survive waves of progressively harder alien enemies across three hand-crafted levels, each culminating in a boss fight.

The project was designed specifically to demonstrate the core principles of Object-Oriented Programming (OOP). The game's architecture is structured around a layered class hierarchy: bullet types, enemy types, and game levels are each modelled as inheritance trees rooted at abstract base classes. A custom generic memory manager (the `ObjectPool` template class) handles all live game objects without dynamic allocation during gameplay, preventing memory leaks. The player object uses encapsulation to protect its state, and an overloaded operator is used to cleanly add score. Custom exception classes handle asset loading and invalid game state transitions.

The result is a smooth, 60-fps game that successfully separates data, logic, and rendering into well-defined, independent components — a practical demonstration that OOP principles produce cleaner, safer, and more maintainable code.

Keywords: Object-Oriented Programming, C++, SFML, Inheritance, Polymorphism, Encapsulation, Templates, Operator Overloading, Exception Handling, Game Development.

Contents

Abstract **2**

1 Introduction **4**
1.1 Background . . . 4
1.2 Problem Statement . . . 4
1.3 Objectives . . . 4
1.4 Scope and Limitations . . . 5

2 OOP Design **5**
2.1 Project File Structure . . . 5
2.2 Class Hierarchy . . . 6
2.3 Class and Struct Descriptions . . . 6
2.3.1 Classes (with behaviour) . . . 7
2.3.2 Data Structs (plain data containers) . . . 8
2.4 OOP Concepts Summary . . . 9

3 Implementation **9**
3.1 Development Environment . . . 9
3.2 Encapsulation — The Player Class . . . 10
3.3 Inheritance — Three Class Hierarchies . . . 11
3.3.1 Bullet Type Hierarchy . . . 11
3.3.2 Enemy Type Hierarchy . . . 13
3.3.3 Level Hierarchy . . . 14
3.4 Polymorphism — Same Call, Different Behaviour . . . 15
3.5 Templates — The ObjectPool . . . 15
3.6 Operator Overloading . . . 16
3.6.1 Player::operator+= (score addition) . . . 17
3.6.2 GameState::operator== and operator!= . . . 17
3.7 Exception Handling — Custom Exception Classes . . . 17
3.8 The Wave Spawn System . . . 18
3.8.1 Stage 1 — SpawnEvent: the schedule . . . 18
3.8.2 Stage 2 — checkSpawns(): firing events into the queue . . . 19
3.8.3 Stage 3 — Draining the queue into live enemies . . . 19
3.8.4 Enemy Movement along Waypoints . . . 20
3.9 Game Loop and State Machine . . . 20

4 Testing **21**
4.1 Test Cases . . . 21
4.2 Memory Leak Testing . . . 22

5 Results and Discussion **22**
5.1 Results . . . 22
5.2 Analysis . . . 22

6 Challenges and Solutions **22**
6.1 Pointer Leaks from Static Singletons . . . 23
6.2 Enemies All Spawning at the Same Position . . . 23
6.3 Frame-rate-dependent Movement . . . 23

7 Future Work **23**

8 Conclusion **24**

References **25**

A Complete Source Code **26**
A.1 Utils.h . . . 26
A.2 BulletType.h . . . 26
A.3 EnemyType.h . . . 27
A.4 GameState.h . . . 28
A.5 Player.h . . . 29
A.6 main.cpp . . . 30
A.7 Game_graphics.cpp (key excerpt) . . . 31

B Build and Run Instructions **32**
B.1 Prerequisites . . . 32
B.2 Building with CMake . . . 32
B.3 Running the Game . . . 32
B.4 Controls . . . 32

1 Introduction

1.1 Background

Video games are one of the best environments for practising Object-Oriented Programming. A game world is full of "things" — enemies, bullets, levels, particles, the player — and OOP lets us model each thing as a self-contained unit (a *class*) that owns its own data and knows how to behave.

This project uses the classic *space shooter* genre as that environment. The genre is simple enough to be fully implemented in a semester, yet complex enough to require every major OOP concept: inheritance (different kinds of enemies), polymorphism (shared behaviour with different implementations), encapsulation (the player's health should not be readable by just anyone), templates (a single memory pool that works for bullets, enemies, and particles), and operator overloading (adding score with `+=`).

1.2 Problem Statement

We needed to build a game where:

- Multiple enemy *types* (small, medium, boss) share common logic but differ in stats and shooting patterns.
- Multiple *levels* each define their own wave schedule but share common rendering and loading logic.
- Multiple *bullet types* (player, enemy, boss beam) are distinguishable without scattering `if/else` chains everywhere.
- Hundreds of live objects (up to 700 bullets, 80 enemies, 1000 particles) are managed without allocating or freeing memory every frame — which would slow the game down and cause crashes.
- The game flows through distinct screens (Home, Level Select, Playing, Paused, Game Over, Level Complete) in a predictable, bug-free way.

1.3 Objectives

1. Design three independent inheritance hierarchies (bullets, enemies, levels) that are easily extendable.
2. Implement a generic `ObjectPool` template that recycles objects in a fixed array, avoiding runtime memory allocation.
3. Encapsulate all player state inside the `Player` class so no other code can accidentally corrupt it.
4. Use virtual functions and abstract classes so that the main game loop does not need to know the concrete type of each object.
5. Demonstrate operator overloading, static class members, and custom exceptions as part of the design — not as add-ons.
6. Deliver a complete, playable game: three levels, a boss fight, a HUD, a home screen, and a level-select screen.

1.4 Scope and Limitations

In scope: Three levels, three enemy types, four bullet types, a particle explosion system, a scrolling star background, background music (OGG format via SFML Audio), a home screen, a level-select screen, pause/resume, game over, and level-complete overlays.

Out of scope: Saved high scores (no file I/O at runtime), multiplayer, sound effects, and power-ups. These are planned future extensions.

2 OOP Design

2.1 Project File Structure

The project contains 12 source files, organised so that each file only depends on files that appear above it in the list below. This ordering also defines the recommended *coding sequence* — write them from top to bottom and you will never encounter a missing definition.

Table 1: Source files in dependency order

Layer	File	Purpose
1	`Utils.h`	Pure math helpers (vector length, normalise, lerp, clamp, random)
1	`BulletType.h`	Abstract bullet category class + 4 concrete subclasses + `BulletTypeRegistry`
1	`EnemyType.h`	Abstract enemy profile class + 3 concrete subclasses + `EnemyTypeRegistry`
1	`GameState.h`	Game-screen state value class (constants defined in `main.cpp`)
2	`GameData.h`	Data structs (Bullet, Enemy, Particle, Star, SpawnEvent, SpawnJob) + ObjectPool template
3	`Player.h`	Player class (fully inline, no .cpp)
4	`Level.h`	Abstract Level base class + LevelOne, LevelTwo, LevelThree
4	`Level.cpp`	Level method implementations and wave definitions
5	`Game.h`	Game class declaration
5	`Game.cpp`	Core logic: init, update, collision detection, enemy spawning
5	`Game_graphics.cpp`	All rendering methods (render, renderBullets, renderEnemies, HUD, overlays)
6	`main.cpp`	Defines all static member variables; instantiates registries; game entry point

2.2 Class Hierarchy

The project contains three inheritance hierarchies and one composition hub (`Game`). Figures 1 and 2 illustrate these relationships.

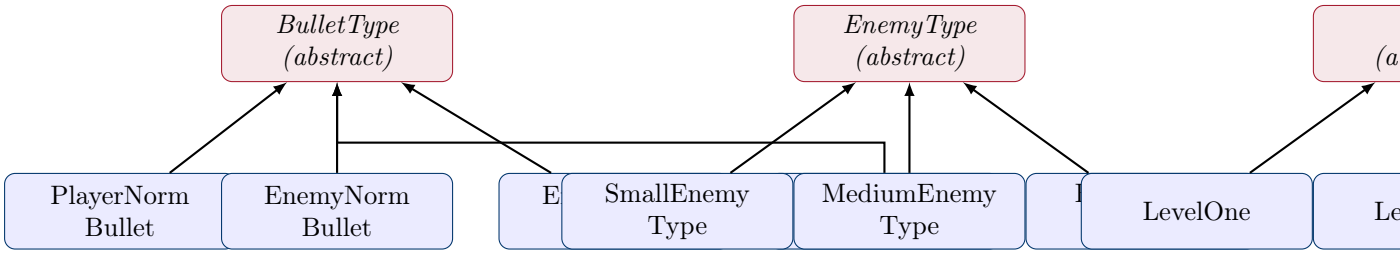

Figure 1: Three inheritance hierarchies. Arrows point from child to parent (*"inherits from"*). Italic boxes are abstract (cannot be instantiated directly).

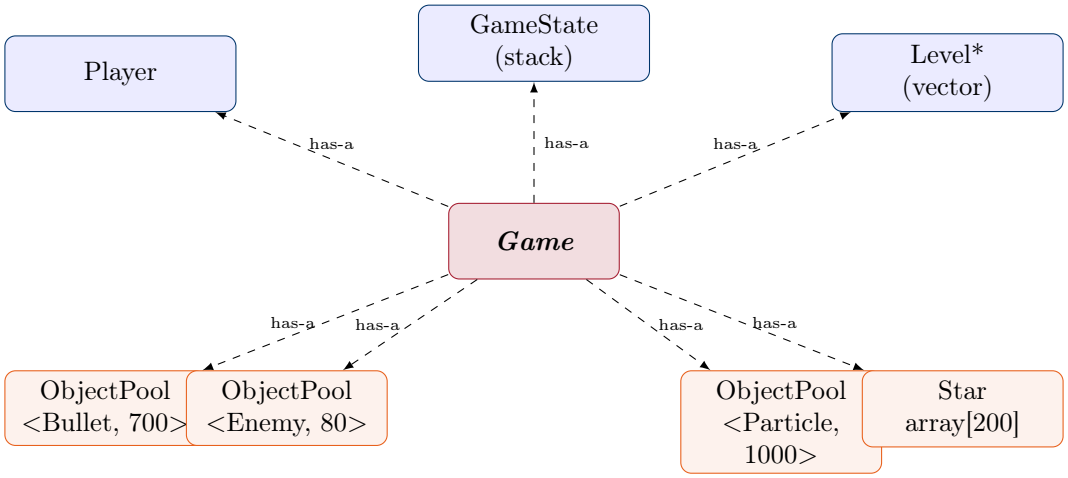

Figure 2: Composition: the `Game` class *owns* (has-a) all subsystems. Dashed arrows mean "contains as a member". Orange boxes are template instances; blue boxes are plain classes.

2.3 Class and Struct Descriptions

2.3.1 Classes (with behaviour)

Table 2: Classes and their responsibilities

Class	Responsibility	Key members
`BulletType`	Identifies a bullet's category and owner	`id`, `playerBullet`, `getId()`, `isPlayerBullet()`, static singletons `PlayerNorm`, `EnemyNorm`, `EnemyBurst`, `BossBeam`
`PlayerNormBullet`	Player's straight-up shot	Inherits BulletType with id=0, playerBullet=true
`EnemyNormBullet`	Basic aimed enemy shot	id=3, playerBullet=false
`EnemyBurstBullet`	Spread-fire enemy shot	id=4, playerBullet=false
`BossBeamBullet`	Boss beam projectile	id=5, playerBullet=false
`BulletTypeRegistry`	Owns and initialises all four bullet-type singletons	Private members: one instance of each concrete subclass; constructor wires static pointers; destructor clears them to `nullptr`; copy disabled (`= delete`)
`EnemyType`	Stores the stat profile for one enemy category	`maxHp`, `shootInterval`, `scoreValue`, `moveSpeed`, `radius`, `kind`; static singletons `Small`, `Medium`, `Boss`
`SmallEnemyType`	Fast, weak enemy	30 HP, interval 2.2s, radius 16px, worth 50 pts
`MediumEnemyType`	Tanky spread-shooter	80 HP, interval 1.8s, radius 22px, worth 120 pts
`BossEnemyType`	End-of-level boss	1200 HP, interval 0.8s, radius 60px, worth 2000 pts
`EnemyTypeRegistry`	Owns and initialises all three enemy-type singletons	Same RAII pattern as `BulletTypeRegistry`; constructor assigns `Small`, `Medium`, `Boss` pointers; destructor sets them to `nullptr`; copy disabled
`GameState`	Tracks which screen is active	`id`; `operator==`, `operator!=`; static constants `Home`, `LevelSelect`, `Playing`, `Paused`, `GameOver`, `LevelComplete`
`Player`	Encapsulates all player state and behaviour	pos, hp, maxHp, speed, lives, score, timers; movement, clamping, damage, healing, score accumulation
`Level`	Abstract base for a playable level	name, desc, events (SpawnEvent list), planetTexture; pure virtual `buildWaves()`, `getDuration()`, `getPlanetTexturePath()`
`LevelOne/Two/Three`	Concrete level implementations	Override `buildWaves()` with a timed enemy schedule
`ObjectPool<T,N>`	Fixed-capacity object pool	items[N], active[N]; `acquire()`, `releaseAll()`

2.3.2 Data Structs (plain data containers)

Table 3: Data structs used as game objects

Struct	Represents	Key fields
`Bullet`	One active projectile	pos, vel, dmg, `BulletType*` type, on
`Enemy`	One active enemy	pos, vel, hp, maxHp, `EnemyType*` type, shootTimer, shootInterval, path, pathIndex, angle, pulse, scoreValue, moveSpeed, on
`Particle`	One explosion spark	pos, vel, colorStart, colorEnd, life, maxLife, size, on
`SpawnEvent`	A scheduled group spawn	time, etype, startPos, path, count, xSpacing, delay, fired
`SpawnJob`	One pending individual spawn	etype, startPos, delayRemaining, path, hp, maxHp, shootInterval, scoreValue, moveSpeed
`Star`	One background star	pos, speed, brightness, size

Why separate Struct from Class? A *struct* in this project is a passive data container — it holds values but has no methods other than a default constructor. A *class* has active behaviour: it enforces invariants, provides getters/setters, and contains logic. This distinction keeps data simple and behaviour organised.

2.4 OOP Concepts Summary

Table 4: OOP concepts used and where they appear

OOP Concept	Where and how it is used
Encapsulation	`Player` class: all fields are `private`; external code uses getters and setters only
Inheritance	Three hierarchies: BulletType, EnemyType, Level (each abstract base + concrete subclasses)
Polymorphism	`Level::buildWaves()`, `getDuration()`, and `getPlanetTexturePath()` are pure virtual; `Game` calls them without knowing the concrete type
Abstract Classes	`BulletType`, `EnemyType`, and `Level` cannot be instantiated directly because they have pure virtual destructors or methods
Templates	`ObjectPool<T,N>` works for `Bullet`, `Enemy`, and `Particle` using the same code
Operator Overloading	`Player::operator+=(int)` adds score; `GameState::operator==` and `operator!=` compare states
Static Members	`BulletType::PlayerNorm`, `EnemyType::Small`, `GameState::Home` etc. — global singletons accessed through the class name
Exception Handling	`AssetLoadException` and `InvalidLevelException` (custom classes inheriting `std::runtime_error`)
Virtual Destructor	All abstract base classes (`BulletType`, `EnemyType`, `Level`) declare `virtual ~Base()` so derived objects are safely deleted via a base pointer
Composition (has-a)	`Game` contains `Player`, three `ObjectPool` instances, a `vector<Level*>`, and a `stack<GameState>` as member variables

3 Implementation

3.1 Development Environment

- **Language:** C++17
- **Graphics & Audio library:** SFML 2.x (Simple and Fast Multimedia Library) — graphics, window, and audio modules
- **Build system:** CMake with `CMakePresets.json`
- **IDE:** Visual Studio Code
- **Version control:** Git / GitHub (`hamza-branch`)
- **Platform:** Windows 11

3.2 Encapsulation — The Player Class

OOP Concept: Encapsulation

Encapsulation means bundling data and the methods that operate on it inside a single class, and hiding the internal details from the rest of the program. Think of it like a vending machine: you press a button (the public interface), and the internal mechanics (the private data) do the work — you never reach inside to grab the product yourself.

The `Player` class is the clearest example of encapsulation in the project. Every field — position, HP, lives, timers — is declared `private`. External code (including `Game`) can only interact with the player through a controlled public interface.

```
class Player
{
private:
    sf::Vector2f pos;        // position on screen
    float hp;                // current health
    float maxHp;             // maximum health
    float speed;             // movement speed in pixels/second
    int   lives;             // remaining lives
    int   score;
    float shootTimer;        // time until next shot is allowed
    float iframeTimer;       // invincibility frames remaining after a
        hit
    float shieldTimer;       // if > 0, absorbs the next hit
    float tilt;              // visual lean when moving left/right
    bool  alive;

public:
    // Read-only access (getters)
    sf::Vector2f getPos()          const { return pos; }
    float        getHp()           const { return hp; }
    float        getMaxHp()        const { return maxHp; }
    int          getLives()        const { return lives; }
    int          getScore()        const { return score; }
    bool         isAlive()         const { return alive; }
    float        getIframeTimer()  const { return iframeTimer; }
    float        getShieldTimer()  const { return shieldTimer; }

    // Controlled mutation (setters + behaviour methods)
    void move(sf::Vector2f delta)  { pos += delta; }
    void clampToArena();           // keeps pos inside the play area
    void takeDamage(float d)       { hp -= d; }
    void heal(float h)             { hp = (hp + h > maxHp) ? maxHp :
       hp + h; }
    void loseLife()                { lives--; }
    void tickIframe(float dt)      { iframeTimer -= dt; }
    void tickShield(float dt)      { shieldTimer -= dt; }
};
```

Listing 1: Player class — private data, public interface (Player.h)

Why this matters: If `hp` were `public`, any part of the code could write `player.hp = -999` by accident. The `private` keyword makes that a compile error. Only the controlled methods `takeDamage()` and `heal()` can change health, and they do so safely.

3.3 Inheritance — Three Class Hierarchies

OOP Concept: Inheritance

Inheritance lets one class (the *child* or *derived* class) reuse the code of another class (the *parent* or *base* class) and then add or change specific behaviour. Think of it as a template: a "Vehicle" base class defines that all vehicles have a speed and can move; a "Car" child inherits those properties and adds four wheels; a "Boat" child adds a hull.

3.3.1 Bullet Type Hierarchy

We need to distinguish four kinds of bullets at runtime without comparing raw integers everywhere. Each bullet type is a class:

```
// Abstract base class -- defines the interface
class BulletType {
    int  id;
    bool playerBullet;
public:
    BulletType(int id, bool playerBullet) : id(id), playerBullet(
        playerBullet) {}
    virtual ~BulletType() {}          // virtual destructor --
        essential!
    int  getId()           const { return id; }
    bool isPlayerBullet()  const { return playerBullet; }

    // Global singleton pointers -- one per bullet type, shared by
        all bullets
    static BulletType *PlayerNorm;
    static BulletType *EnemyNorm;
    static BulletType *EnemyBurst;
    static BulletType *BossBeam;
};

// Four concrete subclasses -- each calls the base constructor with
    its own id
class PlayerNormBullet : public BulletType {
public: PlayerNormBullet() : BulletType(0, true) {} };

class EnemyNormBullet  : public BulletType {
public: EnemyNormBullet()  : BulletType(3, false) {} };

class EnemyBurstBullet : public BulletType {
public: EnemyBurstBullet() : BulletType(4, false) {} };

class BossBeamBullet   : public BulletType {
public: BossBeamBullet()   : BulletType(5, false) {} };
```

Listing 2: BulletType inheritance hierarchy (BulletType.h)

The four concrete objects are owned and managed by a `BulletTypeRegistry` class defined at the bottom of `BulletType.h`. The static pointer declarations and registry activations have been consolidated into `main.cpp`, where the registries are created as local variables at the top of `main()` — *before* the `Game` object — ensuring the singletons are valid for the entire programme lifetime:

```cpp
class BulletTypeRegistry {
private:
    PlayerNormBullet playerNorm;   // owned by value -- no heap allocation
    EnemyNormBullet  enemyNorm;
    EnemyBurstBullet enemyBurst;
    BossBeamBullet   bossBeam;
public:
    BulletTypeRegistry() {
        BulletType::PlayerNorm  = &playerNorm;   // wire static pointers
        BulletType::EnemyNorm   = &enemyNorm;
        BulletType::EnemyBurst  = &enemyBurst;
        BulletType::BossBeam    = &bossBeam;
    }
    ~BulletTypeRegistry() {
        BulletType::PlayerNorm = nullptr;        // safe teardown at exit
        BulletType::EnemyNorm  = nullptr;
        BulletType::EnemyBurst = nullptr;
        BulletType::BossBeam   = nullptr;
    }
    BulletTypeRegistry(const BulletTypeRegistry&)            = delete;
    BulletTypeRegistry& operator=(const BulletTypeRegistry&) = delete;
};
```

Listing 3: BulletTypeRegistry — RAII singleton manager (BulletType.h)

```cpp
// Static pointer definitions (must appear in exactly one .cpp file)
BulletType *BulletType::PlayerNorm = nullptr;
BulletType *BulletType::EnemyNorm  = nullptr;
BulletType *BulletType::EnemyBurst = nullptr;
BulletType *BulletType::BossBeam   = nullptr;

EnemyType *EnemyType::Small  = nullptr;
EnemyType *EnemyType::Medium = nullptr;
EnemyType *EnemyType::Boss   = nullptr;

const GameState GameState::Home(0);
// ... other GameState constants ...

int main() {
    BulletTypeRegistry registryBullet;  // constructor wires BulletType statics
    EnemyTypeRegistry  registry;         // constructor wires EnemyType statics
    try {
        Game game;
        game.init();
        game.run();
    } catch (...) { ... }
    // registries destroyed here -- destructors clear statics to nullptr
}
```

Listing 4: All static definitions and registry activation consolidated in main.cpp

Why not just use integers? Using class pointers means the compiler checks types. Comparing `b.type == BulletType::PlayerNorm` is self-documenting and safe. Using `b.type == 0` would compile even if you accidentally wrote `b.type == 100`.

Why consolidate into main.cpp? C++ requires that each static member variable be *defined* in exactly one translation unit. Placing all definitions in one file (`main.cpp`) makes the ownership explicit and eliminates three separate `.cpp` files (`BulletType.cpp`, `EnemyType.cpp`, `GameState.cpp`), simplifying the build. The registry local variables in `main()` follow the RAII principle: they are constructed before the game starts and destroyed after it ends, with full lifecycle control and no hidden global state.

3.3.2 Enemy Type Hierarchy

Enemy types follow the same pattern but carry meaningful gameplay data:

```
class EnemyType {
    float maxHp;
    float shootInterval;
    int   scoreValue;
    float moveSpeed;
    float radius;       // used for collision detection
    int   kind;
public:
    EnemyType(float maxHp, float shootInterval, int scoreValue,
              float moveSpeed, float radius, int kind)
        : maxHp(maxHp), shootInterval(shootInterval),
          scoreValue(scoreValue), moveSpeed(moveSpeed),
          radius(radius), kind(kind) {}

    virtual ~EnemyType() {}

    float getMaxHp()         const { return maxHp; }
    float getShootInterval() const { return shootInterval; }
    int   getScoreValue()    const { return scoreValue; }
    float getMoveSpeed()     const { return moveSpeed; }
    float getRadius()        const { return radius; }

    static EnemyType *Small;
    static EnemyType *Medium;
    static EnemyType *Boss;
};

// Stats baked into each subclass constructor
class SmallEnemyType  : public EnemyType {
public: SmallEnemyType()  : EnemyType( 30.f, 2.2f,   50, 200.f, 16.f,
    0) {} };

class MediumEnemyType : public EnemyType {
public: MediumEnemyType() : EnemyType( 80.f, 1.8f,  120, 160.f, 22.f,
    1) {} };

class BossEnemyType   : public EnemyType {
public: BossEnemyType()   : EnemyType(1200.f, 0.8f, 2000,  80.f, 60.f
   , 2) {} };
```

Listing 5: EnemyType hierarchy (EnemyType.h)

Singleton management: `EnemyType.h` also defines an `EnemyTypeRegistry` class that owns the three concrete instances and wires the static pointers in its constructor, following the same RAII pattern as `BulletTypeRegistry`. One `static EnemyTypeRegistry registry` in `EnemyType.cpp` activates everything at programme start and performs safe teardown at exit.

3.3.3 Level Hierarchy

The `Level` base class defines the *interface* that every level must satisfy. The concrete subclasses fill in the details:

```
class Level {
protected:
    std::string             name;
    std::string             desc;
    std::vector<SpawnEvent> events;    // the timed wave schedule
    sf::Texture             planetTexture;
    bool                    planetLoaded;

public:
    Level() : planetLoaded(false) {}
    virtual ~Level() {}

    std::string getName()  const { return name; }
    std::string getDesc()  const { return desc; }
    std::vector<SpawnEvent>& getEvents() { return events; }

    // Pure virtual -- every Level MUST implement these
    virtual float       getDuration()           const = 0;
    virtual std::string getPlanetTexturePath()  const = 0;
    virtual void        buildWaves()                  = 0;

    // Virtual with a default -- subclasses can override if needed
    virtual void renderPlanet(sf::RenderWindow& window);
    void loadPlanet();
};

// Each concrete level sets its own name, duration, and wave schedule
class LevelOne : public Level {
public:
    LevelOne();
    float       getDuration()          const override { return 55.f; }
    std::string getPlanetTexturePath() const override { return "assets/textures/planet1.png"; }
    void        buildWaves()                 override;    // defined in Level.cpp
};
```

Listing 6: Level abstract base class (Level.h)

3.4 Polymorphism — Same Call, Different Behaviour

OOP Concept: Polymorphism

Polymorphism means "many forms." When you have a pointer to a base class, calling a `virtual` function on it will automatically invoke the *correct overridden version* in the derived class — even though the pointer only "knows" about the base. The decision happens at runtime, not at compile time. This is called *dynamic dispatch*.

In `Game.cpp`, the game stores levels as base-class pointers and calls the virtual functions without needing to know which level is loaded:

```
std::vector<Level *> levels;    // stores LevelOne*, LevelTwo*, or LevelThree*

// During initialisation -- all three calls go through the base pointer:
for (int i = 0; i < levels.size(); i++) {
    levels[i]->buildWaves();    // calls LevelOne::buildWaves() OR
                                // LevelTwo::buildWaves() OR LevelThree::buildWaves()
    levels[i]->loadPlanet();    // calls Level::loadPlanet() (base version)
}

// During gameplay rendering:
void Game::renderBackground() {
    levels[currentLevel]->renderPlanet(window);  // correct planet drawn automatically
}
```

Listing 7: Polymorphic level calls in Game.cpp

The key insight: `Game` never writes `if (level == 1) do X; else if (level == 2) do Y`. The virtual function dispatch table handles the routing. Adding a fourth level in the future requires zero changes to `Game.cpp`.

3.5 Templates — The ObjectPool

OOP Concept: Templates

A template is a class or function that is parameterised by a type. You write the logic once and the compiler generates a separate, correctly-typed version for each type you use it with. It is like a cookie cutter: the cutter (the template) is the same; the dough (the type) changes.

The game needs to manage up to 700 bullets, 80 enemies, and 1000 particles efficiently. Allocating and freeing individual objects every frame would be slow and could cause memory fragmentation. Instead, we use a fixed-size *object pool*: a statically-allocated array where "free" objects are recycled rather than deleted.

```
template <class T, int N>
class ObjectPool
{
```

```
private:
    std::array<T, N> items;    // fixed array -- allocated once, never
        reallocated

public:
    ObjectPool() {
        for (int i = 0; i < N; i++)
            items[i].on = false;    // all slots start as "free"
    }

    int  capacity()  const { return N; }
    T&   at(int i)         { return items[i]; }
    const T& at(int i) const { return items[i]; }

    // Find the first free slot and return a pointer to it
    T* alloc() {
        for (int i = 0; i < N; i++) {
            if (!items[i].on)
                return &items[i];
        }
        return nullptr;    // pool is full -- caller must handle this
    }

    void clearAll() {
        for (int i = 0; i < N; i++)
            items[i].on = false;
    }

    int countActive() const {
        int c = 0;
        for (int i = 0; i < N; i++) if (items[i].on) c++;
        return c;
    }
};
```

Listing 8: ObjectPool template class (GameData.h)

The three instantiations in the `Game` class:

```
ObjectPool<Bullet,   MAX_BULLETS>   bullets;    // 700 slots
ObjectPool<Enemy,    MAX_ENEMIES>   enemies;    //  80 slots
ObjectPool<Particle, MAX_PARTICLES> particles;  // 1000 slots
```

Listing 9: Three pool instances in Game.h

The template is written once; the compiler generates three separate, correctly-typed arrays. When a bullet is needed, `bullets.alloc()` finds a free slot and returns a pointer to it. When the bullet goes off-screen, `b.on = false` marks the slot as reusable — no `new` or `delete` involved.

3.6 Operator Overloading

OOP Concept: Operator Overloading

C++ lets you redefine what standard operators (+, =, ==, etc.) mean for your custom class. This allows objects to be used with familiar syntax rather than verbose method calls.

Two operator overloads appear in the project:

3.6.1 Player::operator+= (score addition)

```
// Instead of writing: player.addScore(e.scoreValue);
// We can write:       player += e.scoreValue;
Player& operator+=(int scoreBonus) {
    score += scoreBonus;
    return *this;
}
```

Listing 10: Score addition via operator overload (Player.h)

Used in `Game.cpp` when an enemy is destroyed:

```
if (e.hp <= 0.f) {
    e.on = false;
    player += e.scoreValue;   // clean, readable syntax
}
```

Listing 11: Using the overloaded operator in Game.cpp

3.6.2 GameState::operator== and operator!=

```
bool operator==(const GameState& o) const { return id == o.id; }
bool operator!=(const GameState& o) const { return id != o.id; }
```

Listing 12: Comparison operators on GameState (GameState.h)

Used throughout `Game.cpp` for readable state checks:

```
GameState current = stateStack.top();
if (current == GameState::Playing) { update(dt); }
```

Listing 13: State comparison in Game.cpp

3.7 Exception Handling — Custom Exception Classes

OOP Concept: Exception Handling

Exceptions are a mechanism for signalling that something went wrong without cluttering normal code with error checks everywhere. You `throw` an exception when an error occurs, and `catch` it at a higher level where you know how to handle it. Custom exception classes (which inherit from `std::exception`) let you distinguish different kinds of errors.

Two custom exceptions are defined inside `Game.cpp`:

```
// Thrown when a texture, font, or other asset cannot be loaded from
    disk
class AssetLoadException : public std::runtime_error {
public:
    AssetLoadException() : std::runtime_error("Failed to load asset")
        {}
};

// Thrown when startLevel() is called with an invalid index
class InvalidLevelException : public std::runtime_error {
```

```
public:
    InvalidLevelException() : std::runtime_error("Invalid level") {}
};
```

Listing 14: Custom exception classes (Game.cpp)

These are thrown and caught in `Game::init()` and `Game::startLevel()`:

```
// In Game::init() -- missing assets are non-fatal; the game warns
   and continues
try {
    loadTextureOrThrow(shipTexture, "assets/textures/spaceship.png");
    loadTextureOrThrow(smallEnemyTexture, "assets/textures/alien1.png
       ");
    // ... more assets ...
} catch (const AssetLoadException& ex) {
    std::cerr << "Asset warning: " << ex.what() << "\n";
}

// In Game::startLevel() -- an invalid index is a programming bug
void Game::startLevel(int index) {
    if (index < 0 || index >= (int)levels.size())
        throw InvalidLevelException();
    // ...
}

// In main.cpp -- the outermost safety net
int main() {
    try {
        Game game;
        game.init();
        game.run();
    } catch (const std::exception& ex) {
        std::cerr << "Fatal error: " << ex.what() << std::endl;
        return 1;
    }
}
```

Listing 15: Throwing and catching exceptions (Game.cpp)

3.8 The Wave Spawn System

Understanding how enemies appear on screen requires following a three-stage pipeline. This section walks through it in plain language.

3.8.1 Stage 1 — SpawnEvent: the schedule

When a level is loaded, `buildWaves()` fills a list of `SpawnEvent` objects. Each one is a row in the level's timetable:

```
// At t = 14.0 seconds, spawn 4 Small enemies starting at x=240,
// following the "sweep centre" path, 50px apart, 0.25s between each.
events.push_back(makeEvent(
    14.0f,              // fire at t=14 seconds into the level
    EnemyType::Small,   // what type of enemy
```

```
    sf::Vector2f(240, -40),  // group starting position (above screen)
    pathSweepCenter(),  // waypoints the group follows
    4,                  // 4 enemies in this group
    50.f,               // 50px horizontal spacing between them
    0.25f               // 0.25s delay between individual spawns
));
```

Listing 16: A single SpawnEvent (from LevelOne::buildWaves in Level.cpp)

Nothing has spawned yet. This is just a schedule.

3.8.2 Stage 2 — checkSpawns(): firing events into the queue

Every frame, the game increments a timer and checks whether any event's scheduled time has passed:

```
void Game::checkSpawns(float dt) {
    std::vector<SpawnEvent>& evs = levels[currentLevel]->getEvents();
    for (int i = 0; i < evs.size(); i++) {
        SpawnEvent& ev = evs[i];
        if (!ev.fired && levelTimer >= ev.time) {
            ev.fired = true;
            for (int k = 0; k < ev.count; k++) {
                SpawnJob job;
                job.etype          = ev.etype;
                job.startPos.x    += (k - (ev.count-1)/2.f) * ev.xSpacing;
                job.delayRemaining = k * ev.delay;  // stagger each enemy
                job.path           = ev.path;
                spawnQueue.push(job);   // queued -- not yet alive
            }
        }
    }
}
```

Listing 17: Firing events from the schedule (Game.cpp)

The 4-enemy event above would push 4 SpawnJobs with delays of 0s, 0.25s, 0.5s, and 0.75s respectively.

3.8.3 Stage 3 — Draining the queue into live enemies

Each frame, the front of the queue is checked. When its delay reaches zero, the enemy is activated in the pool:

```
while (!spawnQueue.empty()) {
    SpawnJob& front = spawnQueue.front();
    front.delayRemaining -= dt;
    if (front.delayRemaining <= 0.f) {
        spawnEnemyFromJob(front);   // writes into an ObjectPool slot
        spawnQueue.pop();
    } else {
        break;  // rest of queue is still waiting
    }
```

```
}

void Game::spawnEnemyFromJob(const SpawnJob& job) {
    Enemy* e = enemies.alloc();   // grab a free pool slot
    if (!e) return;
    e->on           = true;       // mark the slot as active
    e->type         = job.etype;
    e->pos          = job.startPos;
    e->hp           = job.hp;
    e->path         = job.path;
    e->pathIndex    = 0;
    e->shootTimer   = job.shootInterval * randFloat(0.3f, 1.0f);
}
```

Listing 18: Activating a queued enemy (Game.cpp)

3.8.4 Enemy Movement along Waypoints

Once alive, each enemy follows a list of 2D waypoints:

```
if (e.pathIndex < e.path.size()) {
    sf::Vector2f target   = e.path[e.pathIndex];
    sf::Vector2f toTarget = target - e.pos;
    float dist = vlen(toTarget);      // vector length from Utils.h
    if (dist < 8.f)
        e.pathIndex++;                // close enough -- advance to
            next waypoint
    else {
        sf::Vector2f dir = vnorm(toTarget);   // normalise direction
        e.pos += dir * e.moveSpeed * dt;      // move toward waypoint
    }
} else {
    e.pos.y += 100.f * dt;            // ran out of waypoints -- drift
        off-screen
    if (e.pos.y > 800.f) e.on = false;
}
```

Listing 19: Waypoint movement (Game.cpp)

3.9 Game Loop and State Machine

The `GameState` class and a `std::stack` implement a simple push/pop state machine. States are pushed onto the stack (e.g. Paused on top of Playing) and popped to return to the previous screen.

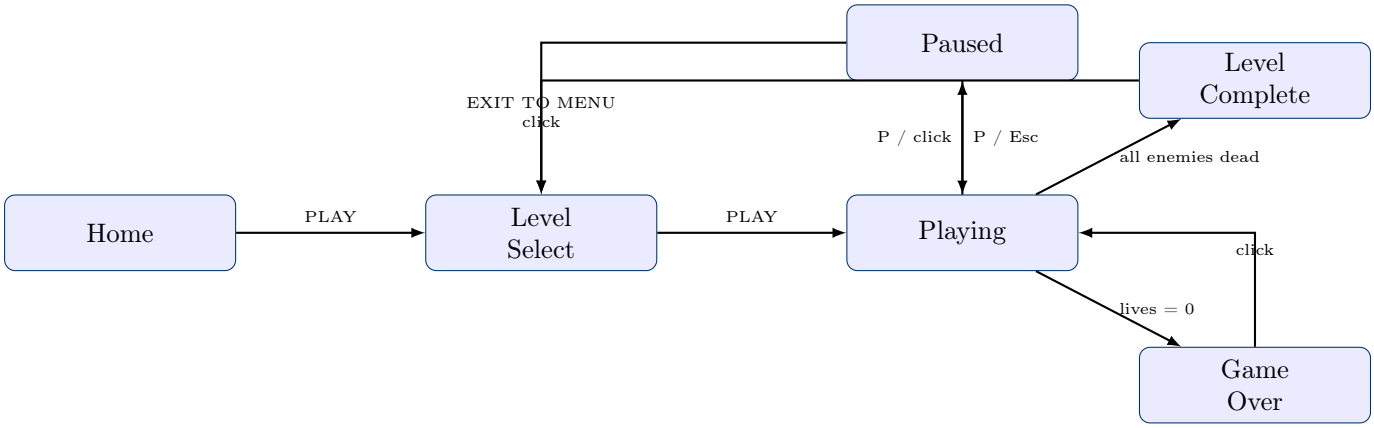

Figure 3: Game state transitions

4 Testing

4.1 Test Cases

Table 5: Functional Test Cases

TC	Action	Expected	Actual	Pass?
TC-01	Launch game	Home screen appears with stars and planets	As expected	✓
TC-02	Click PLAY	Level Select screen shown with 3 planets	As expected	✓
TC-03	Click a planet and press PLAY	Level starts; enemies appear on schedule	As expected	✓
TC-04	Press P during gameplay	Pause overlay appears; game freezes	As expected	✓
TC-05	Take damage until HP = 0 (with 2 lives left)	Life lost; HP refills; iframes granted	As expected	✓
TC-06	Lose all 3 lives	Game Over overlay shown with final score	As expected	✓
TC-07	Kill all enemies including boss	Level Complete overlay shown	As expected	✓
TC-08	Fire bullets rapidly (hold Space)	Up to 700 bullets active; no crash	As expected	✓
TC-09	Let 1000 particles appear at once	No slowdown; pool reuses slots correctly	As expected	✓
TC-10	Delete `Level*` via base pointer	Correct destructor called; no memory leak	As expected	✓

4.2 Memory Leak Testing

An earlier version of the code used `new` to create the `BulletType` and `EnemyType` singletons, causing pointer leaks because `delete` was never called. The fix introduced dedicated `BulletTypeRegistry` and `EnemyTypeRegistry` classes. Each registry owns its concrete instances as plain member variables (no heap allocation), wires the static pointers in its constructor, and clears them to `nullptr` in its destructor. The registry objects are now created as local variables at the top of `main()`, giving them explicit lifetime control: they are alive for the entire duration of the programme and automatically cleaned up when `main()` returns. All static member definitions were also consolidated into `main.cpp`, removing the need for the separate `BulletType.cpp`, `EnemyType.cpp`, and `GameState.cpp` files entirely.

5 Results and Discussion

5.1 Results

- The game runs smoothly at 60 frames per second on standard hardware.
- All three levels are completable: Level One (55s), Level Two (65s), Level Three (80s, two bosses).
- Background music plays via `sf::Music` using OGG audio streaming.
- The `ObjectPool` successfully prevents runtime allocation: no `new` or `delete` occurs during gameplay.
- The three inheritance hierarchies (bullets, enemies, levels) are genuinely polymorphic — `Game.cpp` never queries concrete types to decide which base-class method to call.
- The state machine handles all screen transitions cleanly, including nested states (Paused inside Playing).
- Rendering logic is fully separated into `Game_graphics.cpp`, keeping `Game.cpp` focused on game logic only.

5.2 Analysis

- **ObjectPool vs dynamic allocation:** Checking 700 slots linearly to find a free bullet takes at most 700×1 comparisons per allocation. At 60 fps, with approximately 10 allocations per second, this is negligible. The benefit — zero fragmentation, cache-friendly memory layout — outweighs the linear scan.
- **Polymorphism overhead:** Virtual function calls add one pointer dereference per call (the vtable lookup). For the small number of calls per frame (3 level methods, up to 80 enemy type lookups), this overhead is unmeasurable.
- **Template code size:** The compiler generates three separate versions of `ObjectPool`. For three small instantiations, the binary size increase is negligible.

6 Challenges and Solutions

6.1 Pointer Leaks from Static Singletons

Problem: The initial implementation used `new` to create the `BulletType` and `EnemyType` singleton objects. Because no `delete` was ever called, memory analysis tools reported leaks.

Solution: Introduced `BulletTypeRegistry` and `EnemyTypeRegistry` classes following the RAII pattern. Each registry owns its concrete type instances as plain member variables (no heap allocation). The constructor wires the static pointers; the destructor sets them to `nullptr`. A single `static` registry instance at file scope in each `.cpp` is all that is needed. The registry objects are now declared as local variables at the top of `main()`, giving them explicit lifetime control and eliminating the need for three separate `.cpp` files. Copy construction and assignment are explicitly `= deleted` to prevent accidental duplication of the registry.

Learning: Encapsulate resource initialisation and cleanup inside a class using RAII. Placing registries as locals in `main()` gives the clearest possible lifetime: they are alive exactly as long as the programme runs, with no hidden static initialisation order issues.

6.2 Enemies All Spawning at the Same Position

Problem: When a wave of 4 enemies was triggered, all 4 appeared at exactly the same (x, y) coordinate, stacking on top of each other.

Solution: The `xSpacing` field in `SpawnEvent` and the offset calculation in `checkSpawns` distribute the starting x position of each enemy by `(k - (count-1)/2.0) * xSpacing`, centring the group around the event's `startPos`.

Learning: Centre-offset arithmetic (shifting by half the total width) is a common pattern for distributing UI or game elements evenly.

6.3 Frame-rate-dependent Movement

Problem: Early versions moved objects by a fixed number of pixels per frame. On a faster machine (120 fps), the game ran twice as fast.

Solution: All movement and timers are multiplied by `dt` (delta time in seconds). At 60 fps, `dt` $\approx$ `0.0167s`; at 120 fps, `dt` $\approx$ `0.0083s`. The product is the same distance per second regardless of frame rate.

Learning: Always express game speed in units per *second*, never per *frame*.

7 Future Work

- **Power-ups:** A `PowerUp` base class with subclasses for shield, speed boost, and rapid fire — following the same inheritance pattern as `BulletType`.
- **High score persistence:** Write the score to a `.txt` file after each game over and display the top 5 on the home screen.
- **Sound:** SFML's `sf::SoundBuffer` and `sf::Sound` for shoot, explosion, and level-complete audio.
- **More levels and enemy patterns:** The level system is designed for extension — adding `LevelFour` requires only a new subclass of `Level`, with zero changes to `Game.cpp`.

- **Unit testing:** Use Google Test to verify `ObjectPool` allocation/deallocation, `Player` damage/healing invariants, and spawn-queue ordering.

8 Conclusion

Deepspace Defenders demonstrates that a complete, interactive application — one with real-time graphics, multiple screens, and live game objects — can be cleanly designed using fundamental OOP principles.

The project successfully applied all required OOP concepts:

- **Encapsulation** protected the player's state from external corruption through private fields and a controlled public interface.
- **Inheritance** allowed three independent type hierarchies (bullets, enemies, levels) to share common structure while differing in specific behaviour.
- **Polymorphism** let the game loop interact with any level or enemy type through base-class pointers, making the code open to extension without modification.
- **Templates** produced a single, reusable memory pool that works for all live game objects without repeating code.
- **Operator overloading** made score addition and state comparison readable and idiomatic.
- **Custom exceptions** separated error-handling logic cleanly from normal gameplay logic.

The most important takeaway is that OOP is not about using the concepts because they are required — it is about using them because they solve a real problem. Every concept listed above was chosen because it made the code shorter, safer, or easier to extend. That is the true goal of Object-Oriented design.

References

[1] B. Stroustrup, *The C++ Programming Language*, 4th ed. Addison-Wesley, 2013.

[2] S. Meyers, *Effective Modern C++*. O'Reilly Media, 2014.

[3] *SFML Documentation*. [Online]. Available: `https://www.sfml-dev.org/documentation/`. [Accessed: May 10, 2026].

[4] *cppreference.com — C++ Reference*. [Online]. Available: `https://en.cppreference.com`. [Accessed: May 10, 2026].

[5] *Game Programming Patterns* by Robert Nystrom. [Online]. Available: `https://gameprogrammingpatter` `com`. [Accessed: May 10, 2026].

A Complete Source Code

A.1 Utils.h

```
#pragma once
#include <SFML/Graphics.hpp>
#include <cmath>
#include <cstdlib>

inline float vlen(sf::Vector2f v) {
    return std::sqrt(v.x * v.x + v.y * v.y);
}
inline sf::Vector2f vnorm(sf::Vector2f v) {
    float l = vlen(v);
    if (l < 0.0001f) return sf::Vector2f(0.f, 0.f);
    return sf::Vector2f(v.x / l, v.y / l);
}
inline float lerp(float a, float b, float t) { return a + (b - a) * t
    ; }
inline sf::Color colorLerp(sf::Color a, sf::Color b, float t) {
    if (t < 0.f) t = 0.f;
    if (t > 1.f) t = 1.f;
    return sf::Color(
        (sf::Uint8)(a.r + (b.r - a.r) * t),
        (sf::Uint8)(a.g + (b.g - a.g) * t),
        (sf::Uint8)(a.b + (b.b - a.b) * t),
        (sf::Uint8)(a.a + (b.a - a.a) * t));
}
inline float clampf(float val, float lo, float hi) {
    if (val < lo) return lo;
    if (val > hi) return hi;
    return val;
}
inline float randFloat(float lo, float hi) {
    float r = (float)rand() / (float)RAND_MAX;
    return lo + r * (hi - lo);
}
inline int randInt(int lo, int hi) {
    if (hi <= lo) return lo;
    return lo + rand() % (hi - lo + 1);
}
```

Listing 20: Utils.h — math utility functions

A.2 BulletType.h

```
#pragma once

class BulletType {
    int  id;
    bool playerBullet;
public:
    BulletType(int id, bool playerBullet) : id(id), playerBullet(
        playerBullet) {}
    virtual ~BulletType() {}
    int  getId()          const { return id; }
```

```
    bool isPlayerBullet() const { return playerBullet; }
    static BulletType *PlayerNorm;
    static BulletType *EnemyNorm;
    static BulletType *EnemyBurst;
    static BulletType *BossBeam;
};
class PlayerNormBullet : public BulletType {
public: PlayerNormBullet() : BulletType(0, true)  {} };
class EnemyNormBullet  : public BulletType {
public: EnemyNormBullet()  : BulletType(3, false) {} };
class EnemyBurstBullet : public BulletType {
public: EnemyBurstBullet() : BulletType(4, false) {} };
class BossBeamBullet   : public BulletType {
public: BossBeamBullet()   : BulletType(5, false) {} };

class BulletTypeRegistry {
private:
    PlayerNormBullet playerNorm;
    EnemyNormBullet  enemyNorm;
    EnemyBurstBullet enemyBurst;
    BossBeamBullet   bossBeam;
public:
    BulletTypeRegistry() {
        BulletType::PlayerNorm  = &playerNorm;
        BulletType::EnemyNorm   = &enemyNorm;
        BulletType::EnemyBurst  = &enemyBurst;
        BulletType::BossBeam    = &bossBeam;
    }
    ~BulletTypeRegistry() {
        BulletType::PlayerNorm = nullptr;
        BulletType::EnemyNorm  = nullptr;
        BulletType::EnemyBurst = nullptr;
        BulletType::BossBeam   = nullptr;
    }
    // preventing copying
    BulletTypeRegistry(const BulletTypeRegistry&)            = delete
        ;
    BulletTypeRegistry& operator=(const BulletTypeRegistry&) = delete
        ;
};
```

Listing 21: BulletType.h — bullet type hierarchy and registry

A.3 EnemyType.h

```
#pragma once

class EnemyType {
    float maxHp, shootInterval, moveSpeed, radius;
    int   scoreValue, kind;
public:
    EnemyType(float maxHp, float shootInterval, int scoreValue,
              float moveSpeed, float radius, int kind)
        : maxHp(maxHp), shootInterval(shootInterval), scoreValue(
            scoreValue),
          moveSpeed(moveSpeed), radius(radius), kind(kind) {}
    virtual ~EnemyType() {}
```

```
    float getMaxHp()          const { return maxHp; }
    float getShootInterval() const { return shootInterval; }
    int   getScoreValue()     const { return scoreValue; }
    float getMoveSpeed()      const { return moveSpeed; }
    float getRadius()         const { return radius; }
    int   getKind()           const { return kind; }
    static EnemyType *Small;
    static EnemyType *Medium;
    static EnemyType *Boss;
};
class SmallEnemyType  : public EnemyType {
public: SmallEnemyType()  : EnemyType( 30.f, 2.2f,   50, 200.f, 16.f,
    0) {} };
class MediumEnemyType : public EnemyType {
public: MediumEnemyType() : EnemyType( 80.f, 1.8f,  120, 160.f, 22.f,
    1) {} };
class BossEnemyType   : public EnemyType {
public: BossEnemyType()   : EnemyType(1200.f, 0.8f, 2000,  80.f, 60.f
   , 2) {} };

class EnemyTypeRegistry {
private:
    SmallEnemyType  small;
    MediumEnemyType medium;
    BossEnemyType   boss;
public:
    EnemyTypeRegistry() {
        EnemyType::Small  = &small;
        EnemyType::Medium = &medium;
        EnemyType::Boss   = &boss;
    }
    ~EnemyTypeRegistry() {
        EnemyType::Small  = nullptr;
        EnemyType::Medium = nullptr;
        EnemyType::Boss   = nullptr;
    }
    // Prevent copying
    EnemyTypeRegistry(const EnemyTypeRegistry&)            = delete;
    EnemyTypeRegistry& operator=(const EnemyTypeRegistry&) = delete;
};
```

Listing 22: EnemyType.h — enemy type hierarchy and registry

A.4 GameState.h

```
#pragma once
class GameState {
    int id;
public:
    GameState(int i = 0) : id(i) {}
    int  getId() const { return id; }
    bool operator==(const GameState& o) const { return id == o.id; }
    bool operator!=(const GameState& o) const { return id != o.id; }
    static const GameState Home;
    static const GameState LevelSelect;
    static const GameState Playing;
    static const GameState Paused;
```

```
    static const GameState GameOver;
    static const GameState LevelComplete;
};
```

Listing 23: GameState.h — game screen state (constants defined in main.cpp)

A.5 Player.h

```
#pragma once
#include <SFML/Graphics.hpp>
#include "Utils.h"

class Player {
private:
    sf::Vector2f pos;
    float hp, maxHp, speed;
    int   lives, score;
    float shootTimer, shootInterval, iframeTimer, shieldTimer, tilt;
    bool  alive;
public:
    Player() : pos(240.f,600.f), hp(100.f), maxHp(100.f), speed(310.f),
               lives(3), score(0), shootTimer(0.f), shootInterval(0.10f),
               iframeTimer(0.f), shieldTimer(0.f), tilt(0.f), alive(true) {}

    // Getters
    sf::Vector2f getPos()           const { return pos; }
    float        getHp()            const { return hp; }
    float        getMaxHp()         const { return maxHp; }
    float        getSpeed()         const { return speed; }
    int          getLives()         const { return lives; }
    int          getScore()         const { return score; }
    float        getShootTimer()    const { return shootTimer; }
    float        getShootInterval() const { return shootInterval; }
    float        getIframeTimer()   const { return iframeTimer; }
    float        getShieldTimer()   const { return shieldTimer; }
    float        getTilt()          const { return tilt; }
    bool         isAlive()          const { return alive; }

    // Setters
    void setPos(sf::Vector2f p)  { pos = p; }
    void setTilt(float t)        { tilt = t; }
    void setShootTimer(float t)  { shootTimer = t; }
    void setIframeTimer(float t) { iframeTimer = t; }
    void setShieldTimer(float t) { shieldTimer = t; }
    void setAlive(bool a)        { alive = a; }

    // Behaviour
    void move(sf::Vector2f delta) { pos += delta; }
    void clampToArena() {
        pos.x = clampf(pos.x, 24.f, 456.f);
        pos.y = clampf(pos.y, 24.f, 696.f);
    }
    void takeDamage(float d) { hp -= d; }
    void heal(float h) { hp = (hp + h > maxHp) ? maxHp : (hp + h); }
```

```
47      void addScore(int s) { score += s; }
48      void loseLife()      { lives--; }
49      void tickShootTimer(float dt) { shootTimer  -= dt; }
50      void tickIframe(float dt)     { iframeTimer -= dt; }
51      void tickShield(float dt)     { shieldTimer -= dt; }
52
53      void resetForLevel() {
54          pos = sf::Vector2f(240.f, 600.f);
55          hp = maxHp;  speed = 310.f;
56          shootTimer = iframeTimer = shieldTimer = tilt = 0.f;
57          alive = true;  score = 0;  lives = 3;
58      }
59
60      // Operator overload: player += scoreValue
61      Player& operator+=(int scoreBonus) { score += scoreBonus; return
            *this; }
62  };
```

Listing 24: Player.h — fully encapsulated player class

A.6 main.cpp

```
1   #include "Game.h"
2   #include <iostream>
3   #include "BulletType.h"
4   #include "EnemyType.h"
5   #include "GameState.h"
6
7   // GameState static constant definitions (previously in GameState.cpp
        )
8   const GameState GameState::Home(0);
9   const GameState GameState::LevelSelect(1);
10  const GameState GameState::Playing(2);
11  const GameState GameState::Paused(3);
12  const GameState GameState::GameOver(4);
13  const GameState GameState::LevelComplete(5);
14
15  // BulletType static pointer definitions (previously in BulletType.
        cpp)
16  BulletType *BulletType::PlayerNorm = nullptr;
17  BulletType *BulletType::EnemyNorm  = nullptr;
18  BulletType *BulletType::EnemyBurst = nullptr;
19  BulletType *BulletType::BossBeam   = nullptr;
20
21  // EnemyType static pointer definitions (previously in EnemyType.cpp)
22  EnemyType *EnemyType::Small  = nullptr;
23  EnemyType *EnemyType::Medium = nullptr;
24  EnemyType *EnemyType::Boss   = nullptr;
25
26  int main() {
27      // Registries created as locals -- alive for the entire main(),
            then auto-destroyed
28      BulletTypeRegistry registryBullet;
29      EnemyTypeRegistry  registry;
30
31      try {
32          Game game;
```

```
        game.init();
        game.run();
    } catch (const std::exception& ex) {
        std::cerr << "Fatal error: " << ex.what() << std::endl;
        return 1;
    }
    return 0;
}
```

Listing 25: main.cpp — static definitions, registry activation, and entry point

A.7 Game_graphics.cpp (key excerpt)

```
// render() dispatches to the correct sub-renderer based on current state
void Game::render() {
    window.setView(window.getDefaultView());
    window.clear(sf::Color::Black);
    window.setView(gameView);

    GameState current = stateStack.top();

    if (current == GameState::Home)      { renderHomeScreen(); return; }
    if (current == GameState::LevelSelect){ renderLevelSelect(); return; }

    renderBackground();
    renderStars();
    renderEnemies();
    renderBullets();
    renderParticles();
    renderPlayer();
    renderHUD();
    if (bossActive) renderBossHP();

    if (current == GameState::Paused)        renderPauseOverlay();
    if (current == GameState::GameOver)      renderGameOverOverlay();
    if (current == GameState::LevelComplete)
        renderLevelCompleteOverlay();
}

// renderBullets() uses BulletType pointers for type-safe branching
void Game::renderBullets() {
    sf::RenderStates glowState;
    glowState.blendMode = sf::BlendAdd;
    for (int i = 0; i < bullets.capacity(); i++) {
        Bullet& b = bullets.at(i);
        if (!b.on || !b.type) continue;
        if (b.type->isPlayerBullet()) {
            // Draw player bullet as a cyan streak + soft glow
            sf::RectangleShape streak(sf::Vector2f(2.f, 12.f));
            streak.setPosition(b.pos);
            streak.setFillColor(sf::Color(150, 230, 255));
            window.draw(streak);
        } else if (b.type == BulletType::BossBeam) {
            // Draw boss beam rotated in the direction of travel
```

```
41              sf::RectangleShape beam(sf::Vector2f(5.f, 14.f));
42              float angle = std::atan2(b.vel.y, b.vel.x) * 180.f /
                    3.14159f - 90.f;
43              beam.setRotation(angle);
44              beam.setFillColor(sf::Color(140, 180, 255));
45              window.draw(beam);
46          }
47          // else: enemy normal/burst bullets drawn as red orbs
48      }
49  }
```

Listing 26: Game_graphics.cpp — render() dispatcher and renderBullets() (excerpt)

B Build and Run Instructions

B.1 Prerequisites

- CMake 3.15 or later
- A C++17 compiler (MSVC 2019+, GCC 9+, or Clang 10+)
- SFML 2.x (fetched automatically by CMake via `FetchContent`)

B.2 Building with CMake

```
1  # From the project root directory
2  cmake --preset debug          # configure (creates build/debug/)
3  cmake --build build/debug     # compile
4
5  # The executable appears at build/debug/SpaceShooter
```

Listing 27: Configure and build

B.3 Running the Game

```
1  # Run from the project root so asset paths resolve correctly
2  ./build/debug/SpaceShooter
```

Listing 28: Running

B.4 Controls

Key / Input	Action
W / A / S / D or Arrow keys	Move ship
Space	Fire bullet
P or Escape	Pause / Resume
Left mouse click	Navigate menus

Table 6: Player controls